User Manual - Clevis Vector Graphics Software

Introduction

Command Line Vector Graphics Software (Clevis) is a Java-based application that enables users to create, manipulate and manage vector graphics through both Command Line Interface (CLI) and Graphical User Interface (GUI). This software supports shapes such as line segments, circles, rectangles and squares, with advanced features like grouping and transformation, all visualised in real-time.

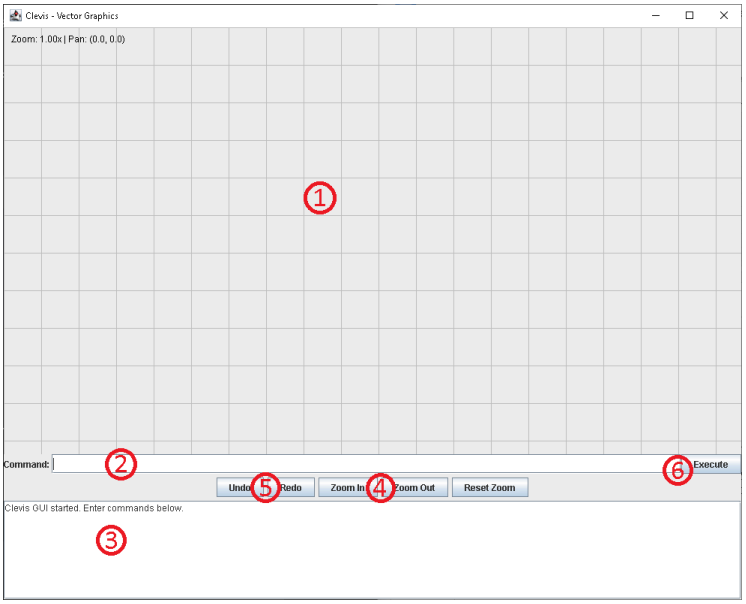
Main features

- **Dual Interface Modes**
  - Full support for Command Line and Graphical Interface
- **Shape Creation & Manipulation**
  - Create shapes such as the circle, rectangle, square, or line, each with unique naming.
  - Manipulate such shapes through operations such as grouping, ungrouping, moving, and deletion
- **Geometric Analysis Operations**
  - Bounding box calculation, intersection detection and point coverage
- **History Management Capabilities**
  - Undo/Redo functionality for any possible combination of shape modification
  - Does not include commands such as boundingbox, intersect, list, listAll, quit
- **Session Logging**
  - Commands are automatically logged and turned into HTML and text files for record keeping purposes
- **Zoom Capabities**
  - For the user's optimal viewing experience, dynamic zooming has been implemented

System Requirements

Java SE Development Kit 21
512 Mb RAM (Minimum)
1074x768 screen resolution or above recommended

Interface



Graphical User Interface:

1. Drawing Canvas
2. Command Input Field
3. Output Console
4. Zoom Controls
5. Undo/Redo Buttons
6. Tool Bar

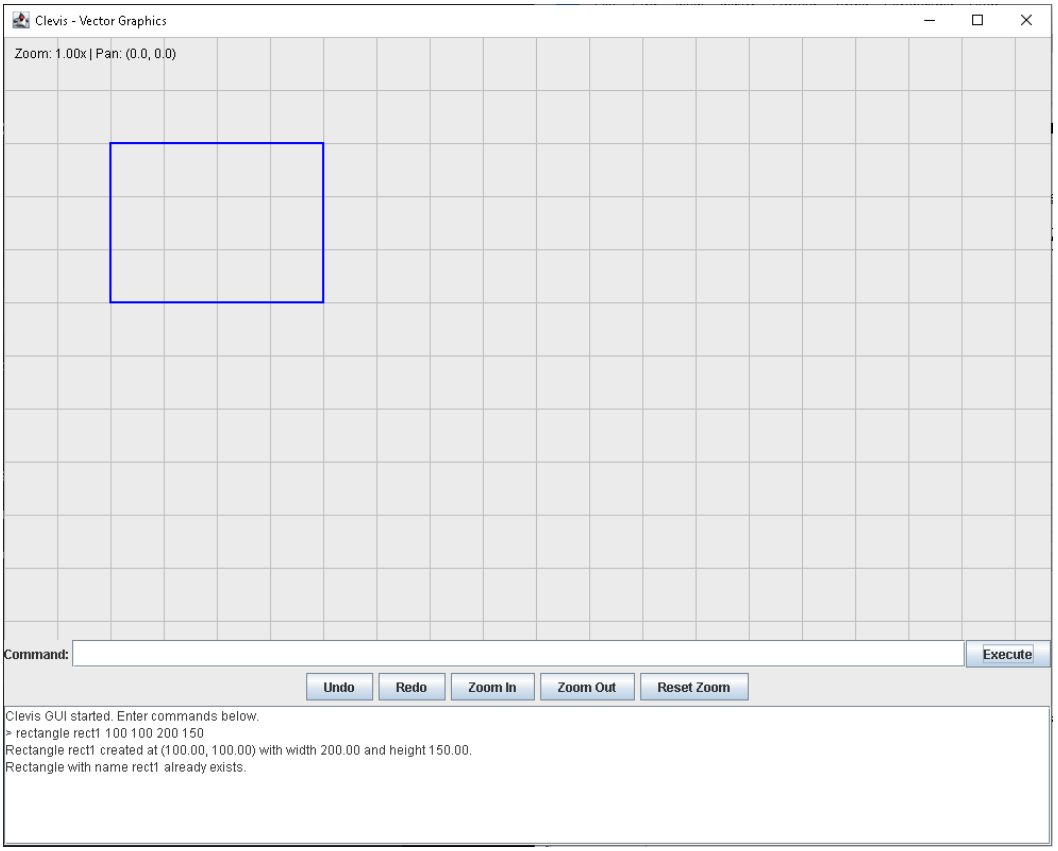
Command Line Interface

Text based interface accepts commands and gives feedback immediately

Comprehensive Command Reference

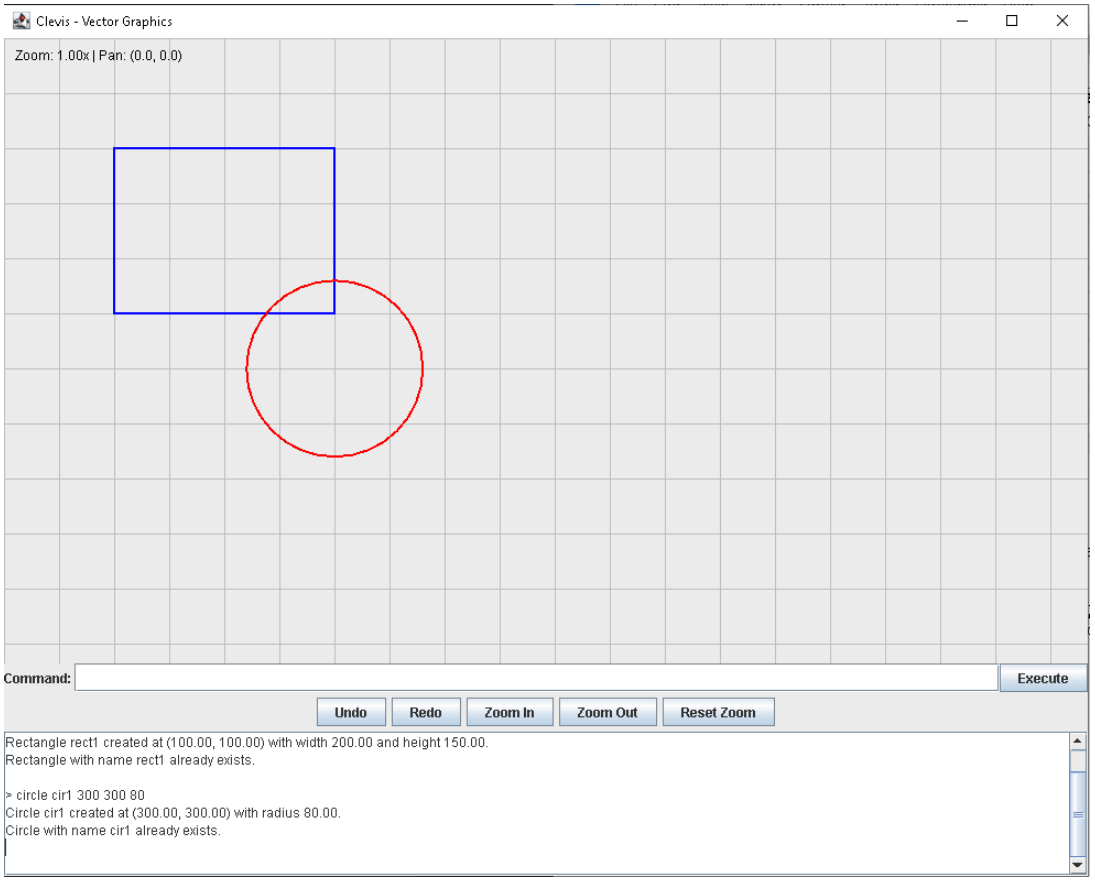
- Shape Creation
  Command: **[shape type] [name of shape] [attributes]**

  Create rectangle:**rectangle [name] [x] [y] [width] [height]**
  Note: (x,y) is the coordinate of top-left corner of the rectangle



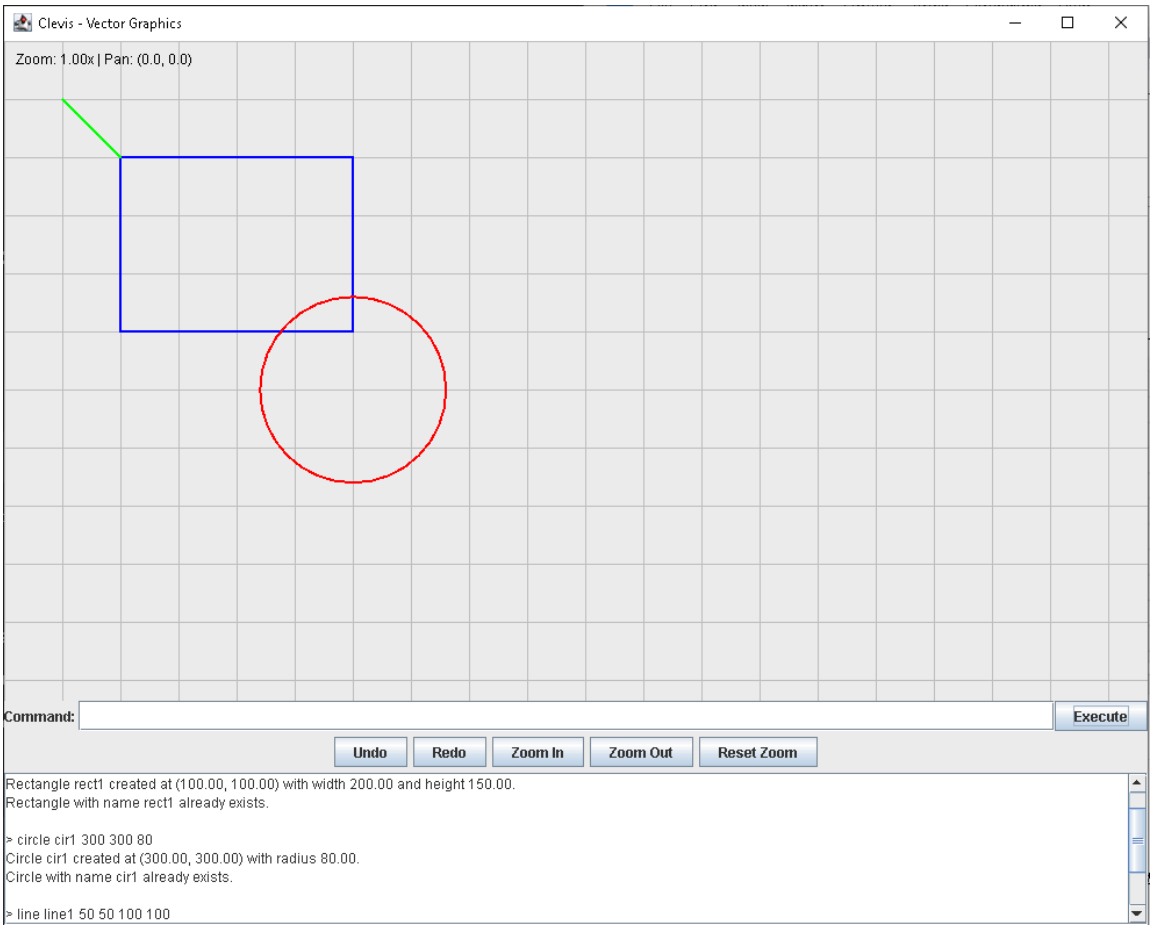
  Command: rectangle rect1 100 100 200 150
  Result: Creates a rectangle named "rect1" at position (100,100) with width 200 and height 150, coloured blue (set as default colour of rectangle to easily distinguish from other shapes)

Create circle: **circle [name] [x] [y] [radius]**
Note: (x,y) is the coordinate of the circle's center point



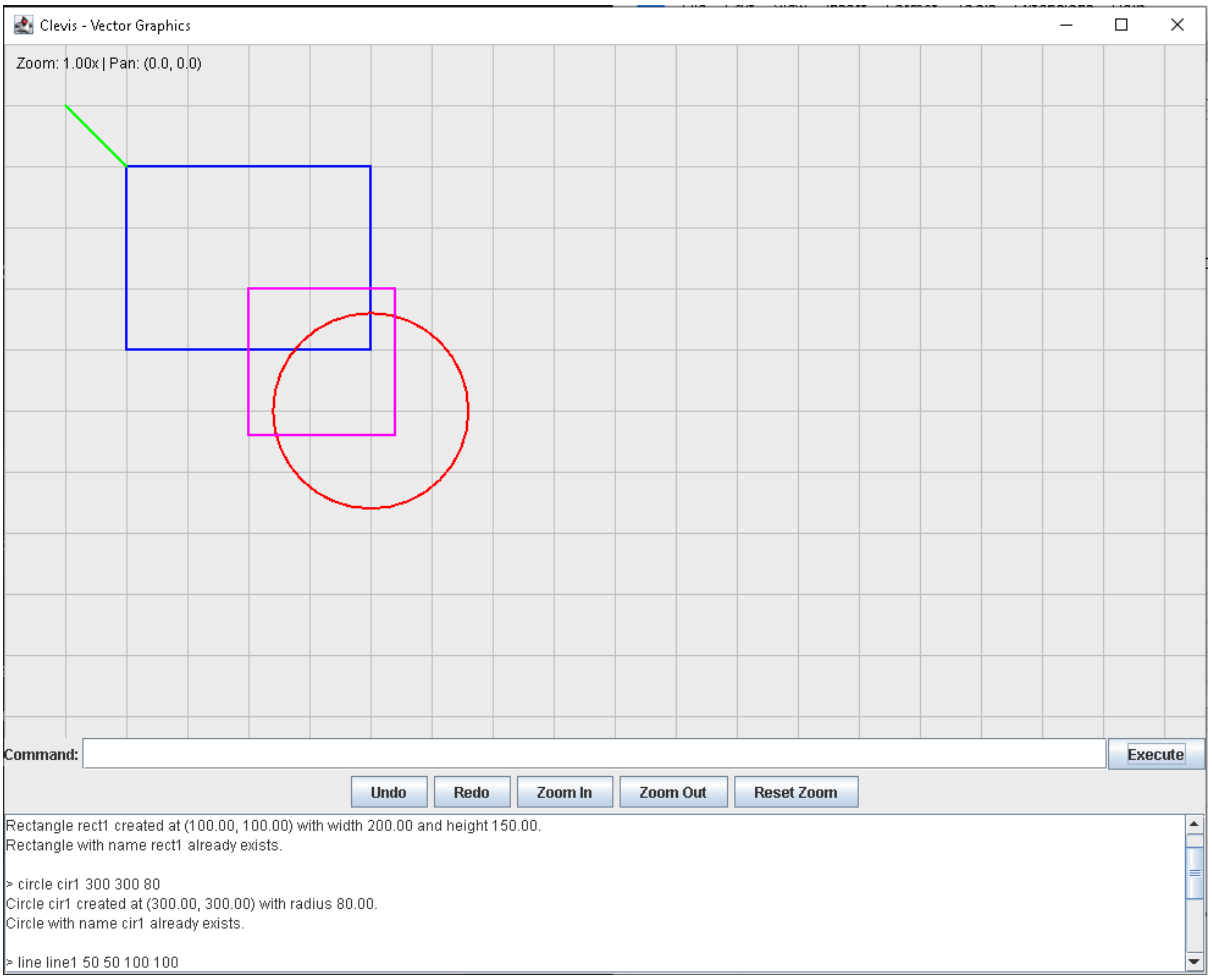
Command: circle cir1 300 300 80
Result: Creates a circle named "cir1" centered at (300,300) with radius of 80 coloured red (set as default colour)

Create line: **line [name] [x1] [y1] [x2] [y2]**
Note: (x1,y1) and (x2,y2) are the coordinate of the 2 ends



Command: line line1 50 50 100 100
Result: Creates a line named "line1" from (50,50) to (100,100) coloured green (set as default colour)

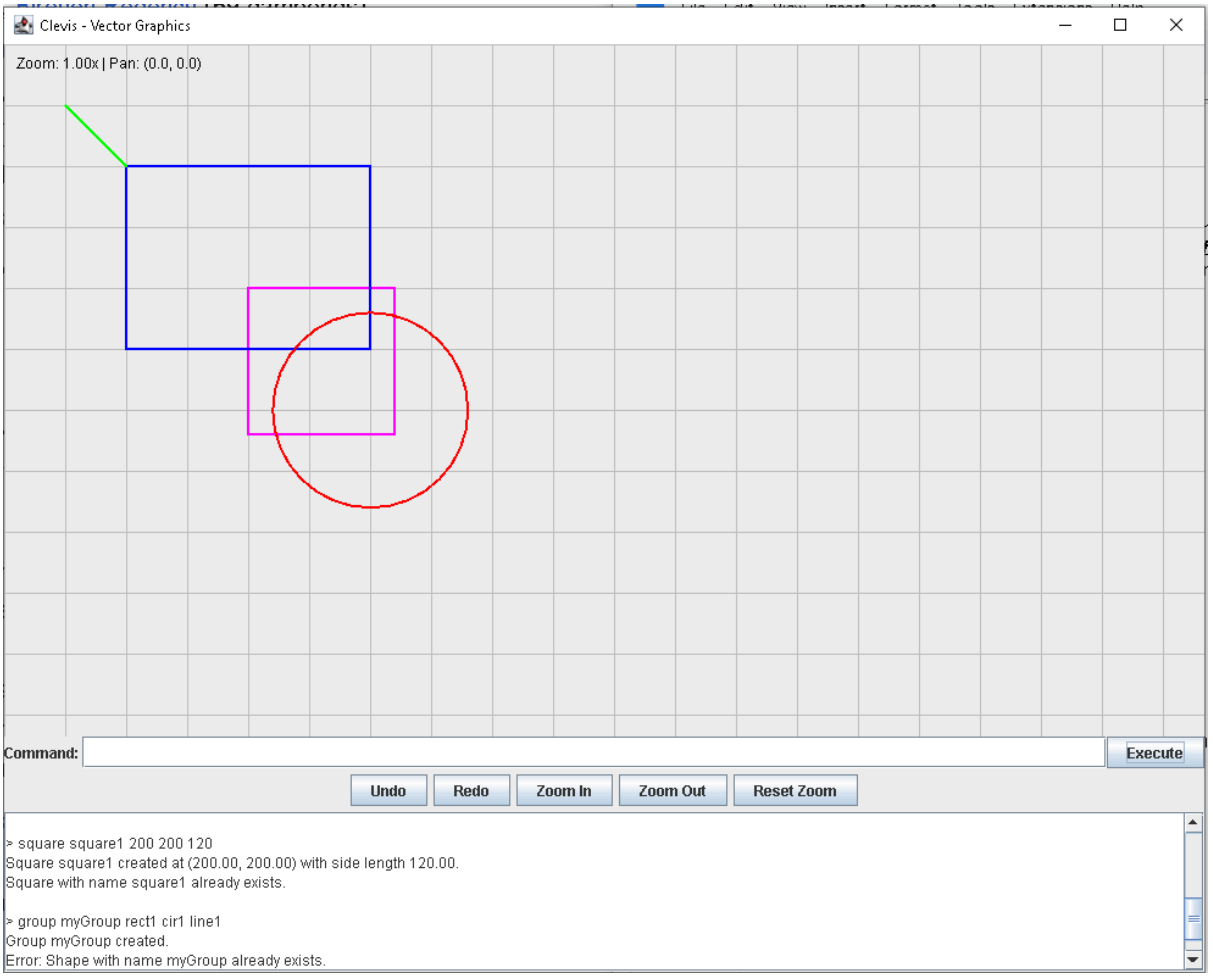
Create square: **square [name] [x] [y] [side]**
Note: (x,y) is the coordinate of top-left corner of the square



Command: square square1 200 200 120
Result: Creates a square named "square1" at (200,200) with side length 120, coloured magenta (set as default colour)

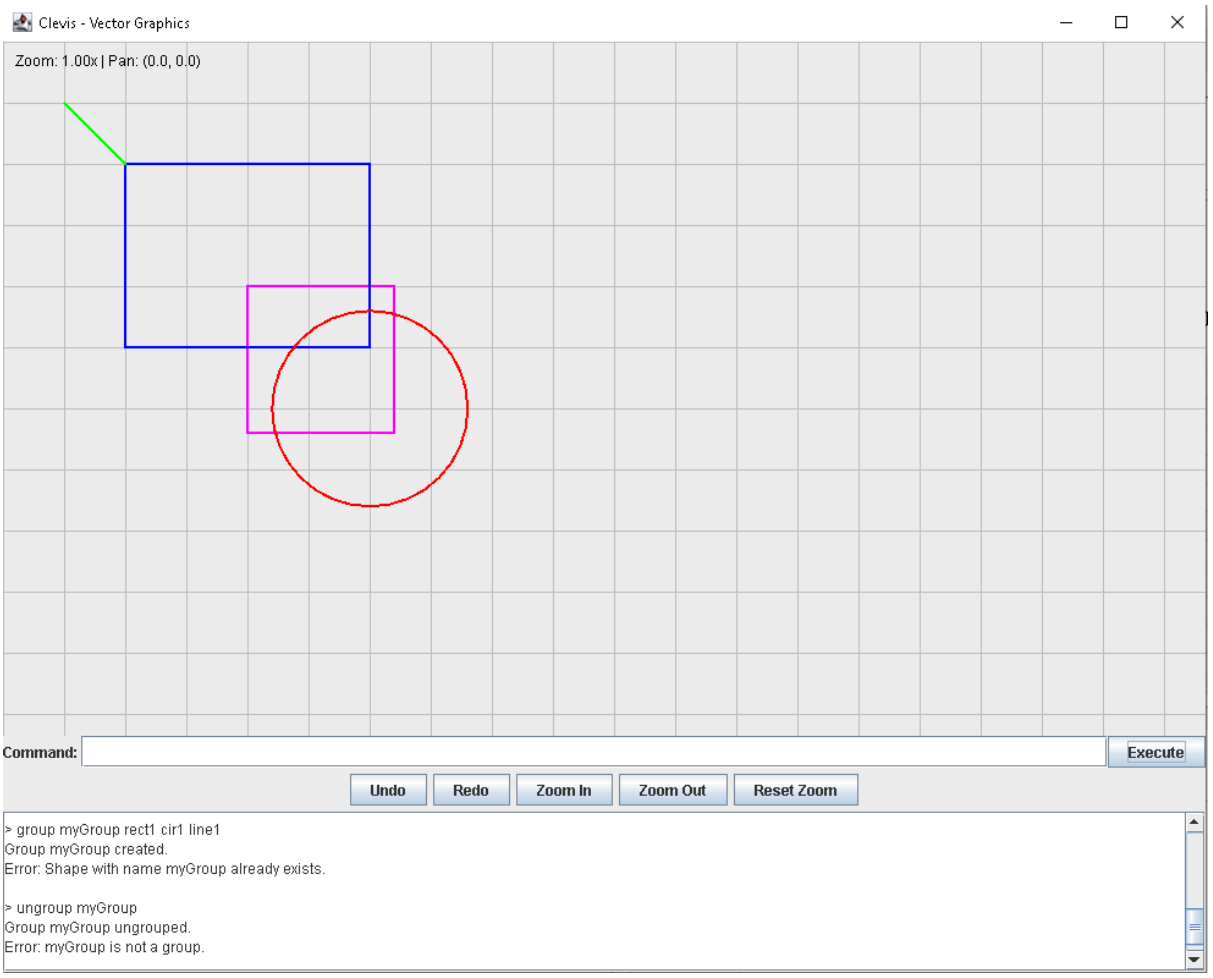
- Shape Manipulation I - Grouping/Ungrouping
  Group Shapes**: group [name of group] [shape1] [shape2] … [shape n]**

  Note: shape1, shape2, shape n are the names of shapes to group



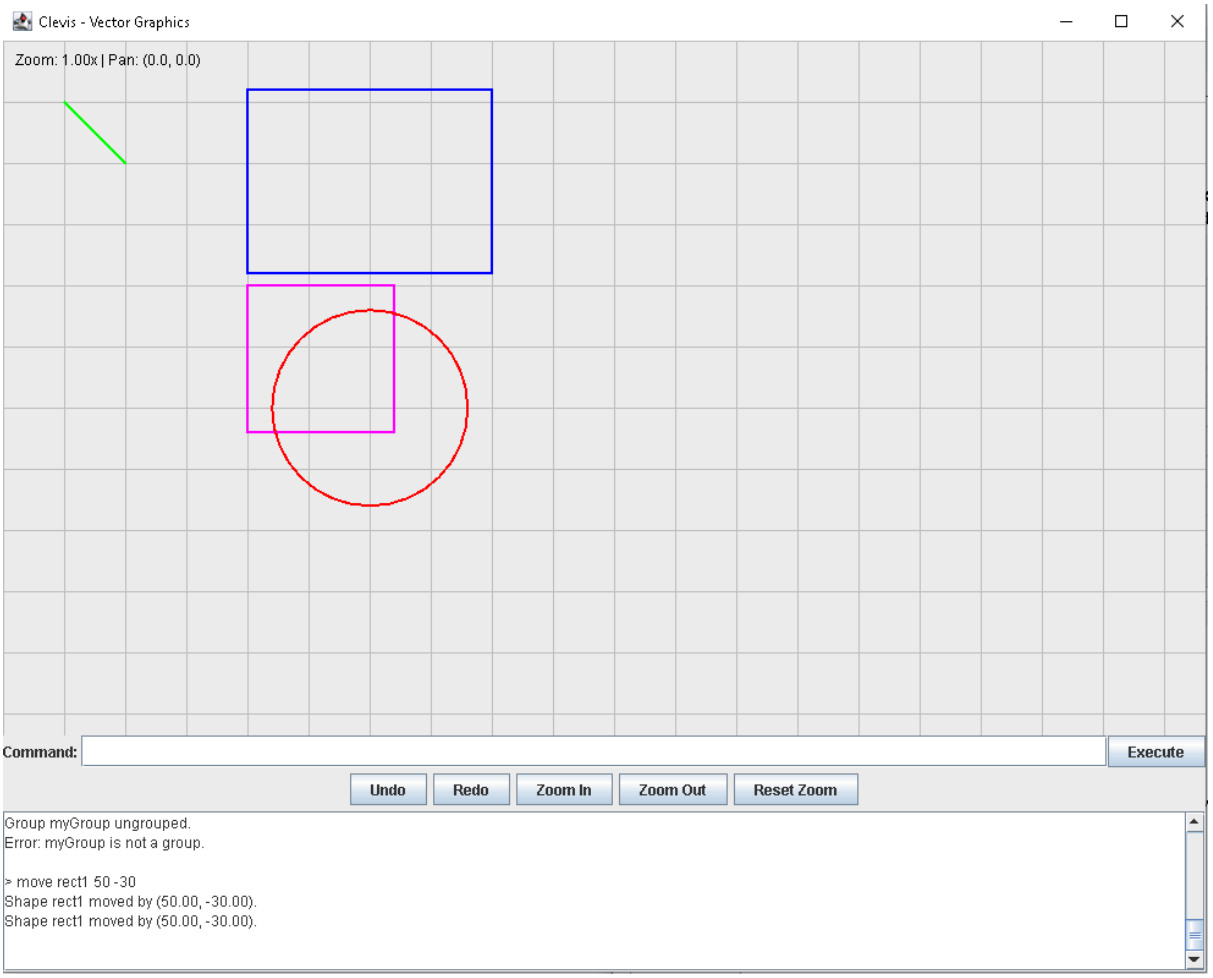
Command: group myGroup rect1 cir1 line1
Result: Groups rectangle "rect1", circle "cir1", and line "line1" into a group named "myGroup"

Ungroup a Group**: ungroup [name of group]**



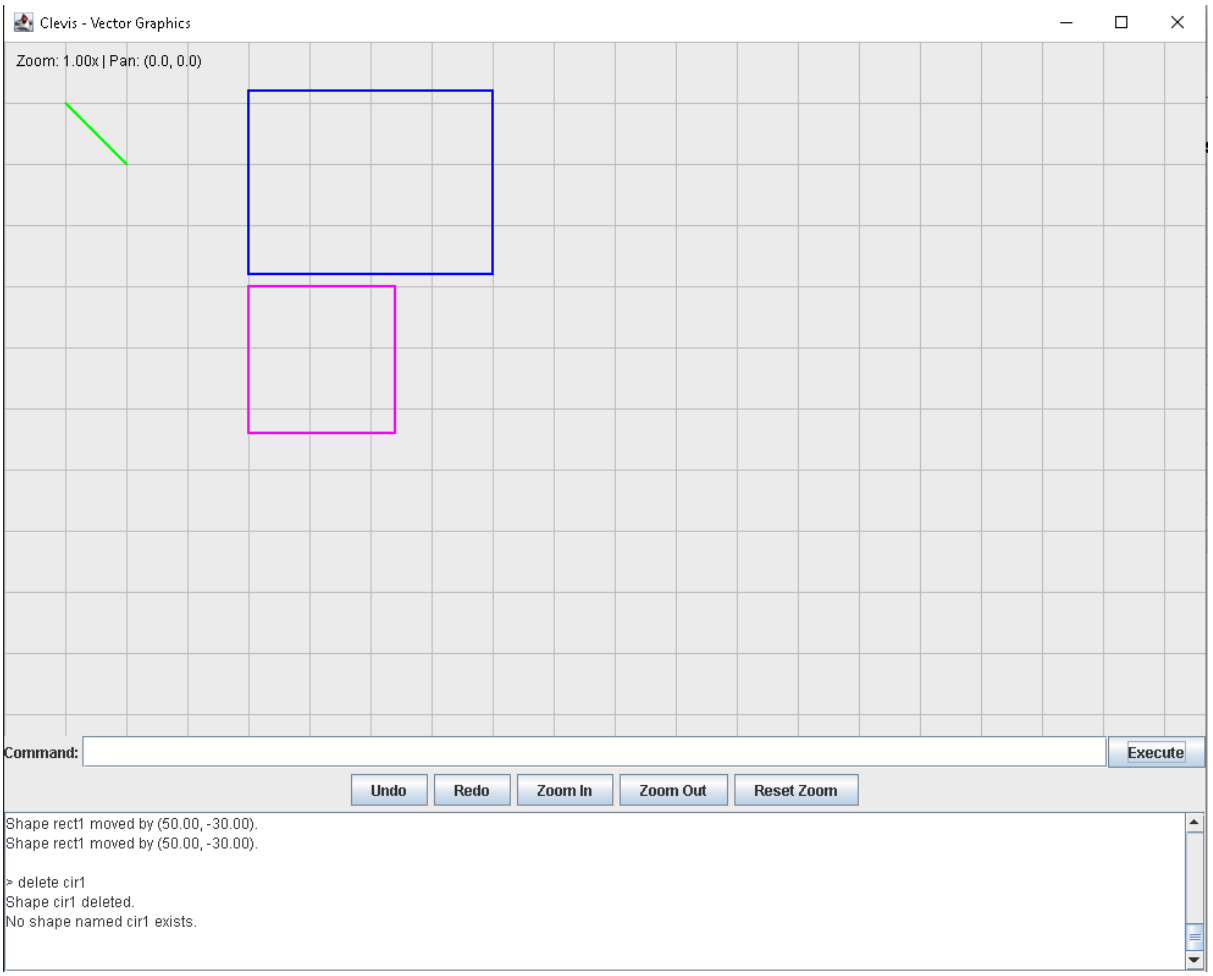
Command: ungroup myGroup
Result: Ungroups "myGroup" back into individual shapes

- Shape Manipulation II - Move & Delete
  Move shape/group: **move [name of shape/ group] [x] [y]**
  Note: x,y are the horizontal and vertical displacement



Command: move rect1 50 -30
Result: move “rect1” 50 units right and 30 units up

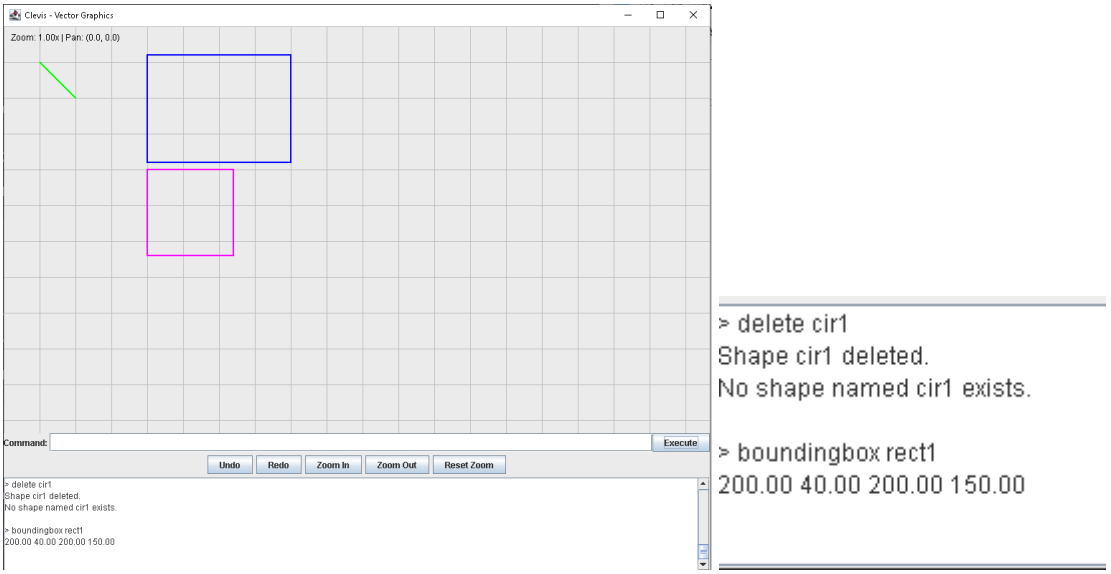
Delete shape/group: **delete [name of shape/group]**



Command: delete cir1
Result: delete circle named “cir1”

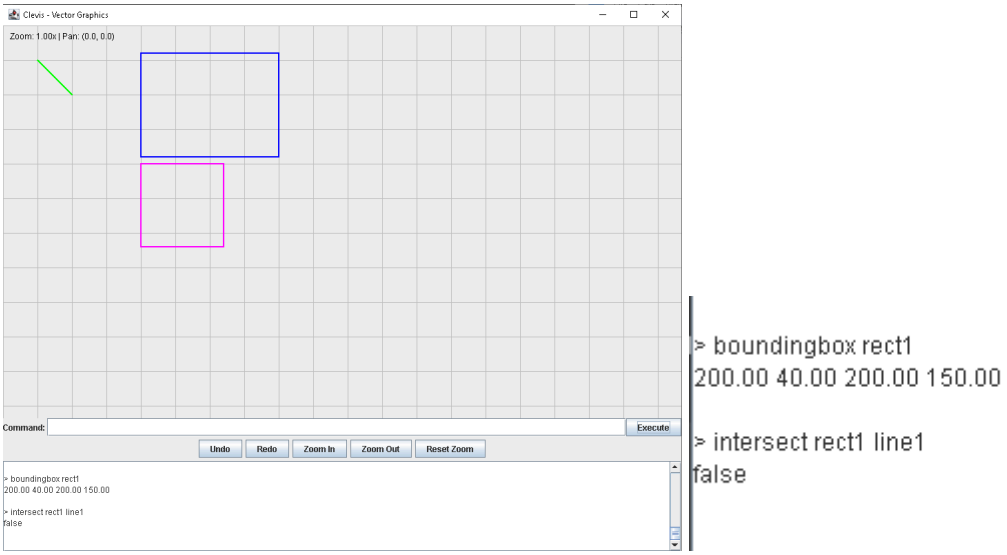
- Analysis and Information Commands

  Bounding box: Display the bounding box coordinates of a shape
  command: **boundingbox [name of shape/group]**



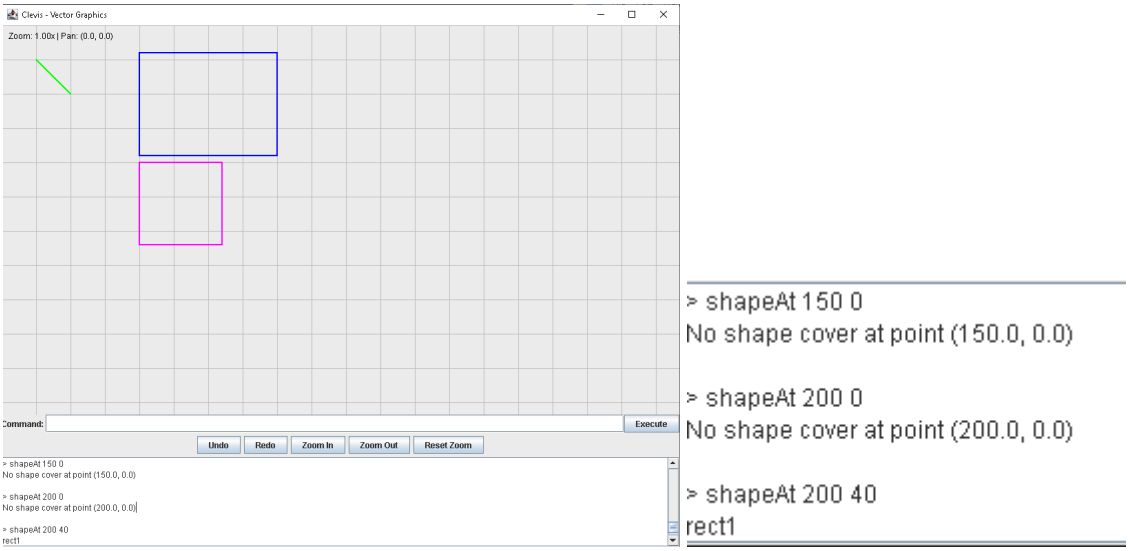
Command: boundingbox rect1
Output: "200.00 40.00 200.00 150.00" (x y width height)

  Intersect: Check if 2 shapes intersect
  command: **intersect [name1] [name2]**



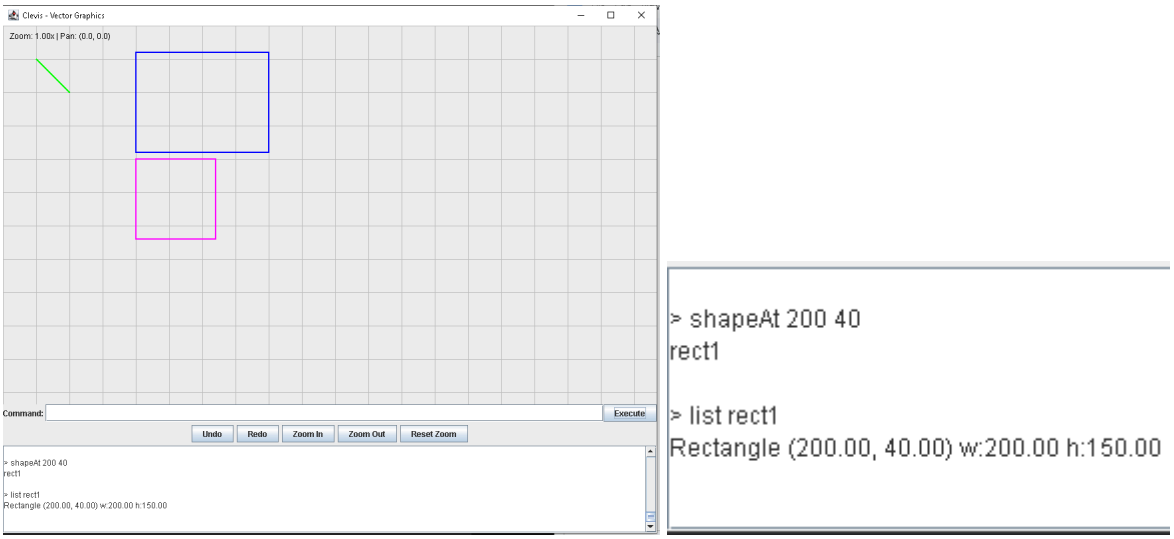
Command: intersect rect1 line1 (check if “line1” and “rect1” intersect)
Output: false

shapeAt: find the top-most shape at specified coordinates
command: **shapeAt [x] [y]**
Note: (x,y): coordinate to check



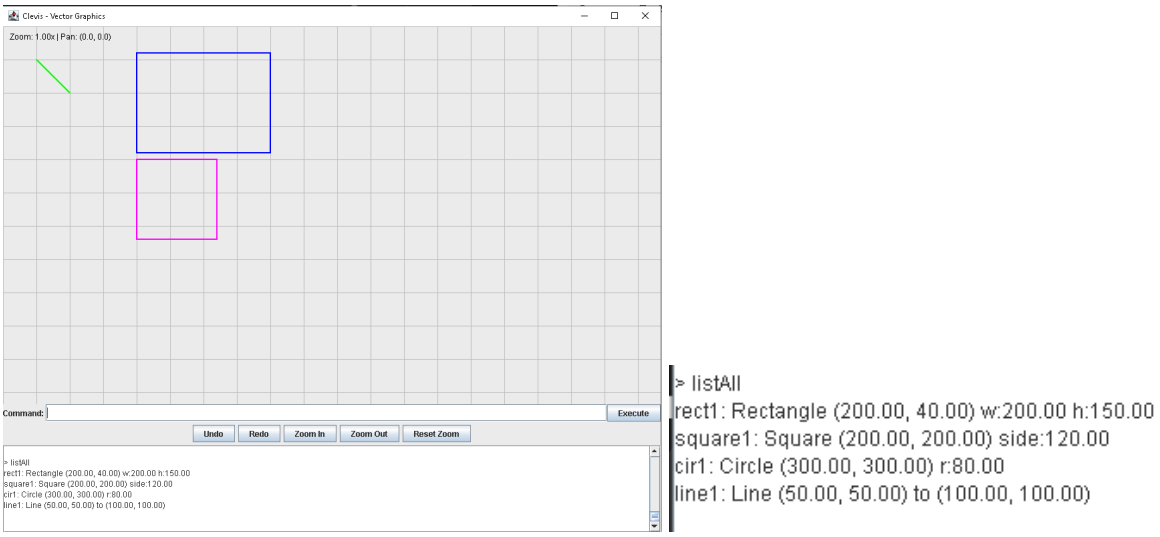
Command: shapeAt 150 0 (check any shape at “150 00”)
Output: No shape cover at point (150.0, 0.0)
Command: shapeAt 200 40 (check any shape at 200 (x) 40 (y))
Output: rect1

List: Display detailed information about specific shape
command: **list (name)**



Command: list rect1
Output:: Rectangle (200.00, 40.00) w:200.00 h:150.00"

List All: Display information about all shapes
command: **listAll**

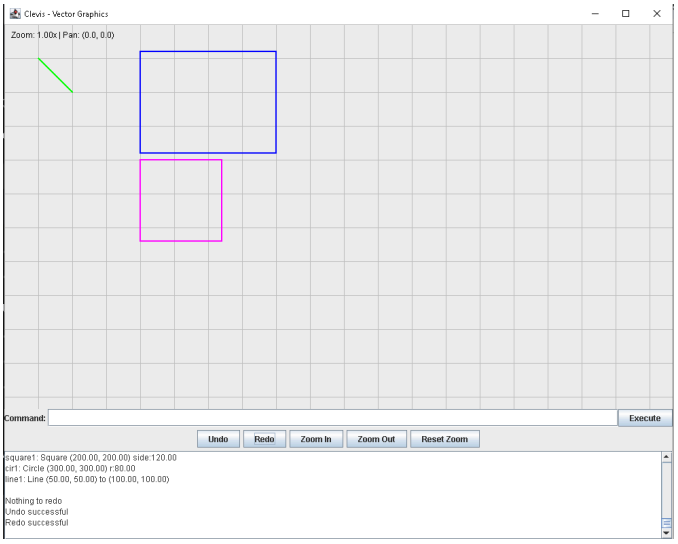


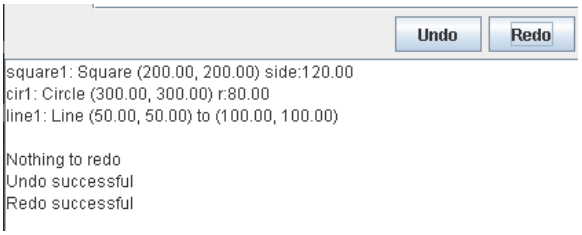
Command: listAll

Output:rect1: Rectangle (200.00, 40.00) w:200.00 h:150.00
square1: Square (200.00, 200.00) side:120.00
cir1: Circle(300.00, 300.00) r:80.00
line1: Line (50.00, 50.00) to (100.00, 100.00)

- Session Management

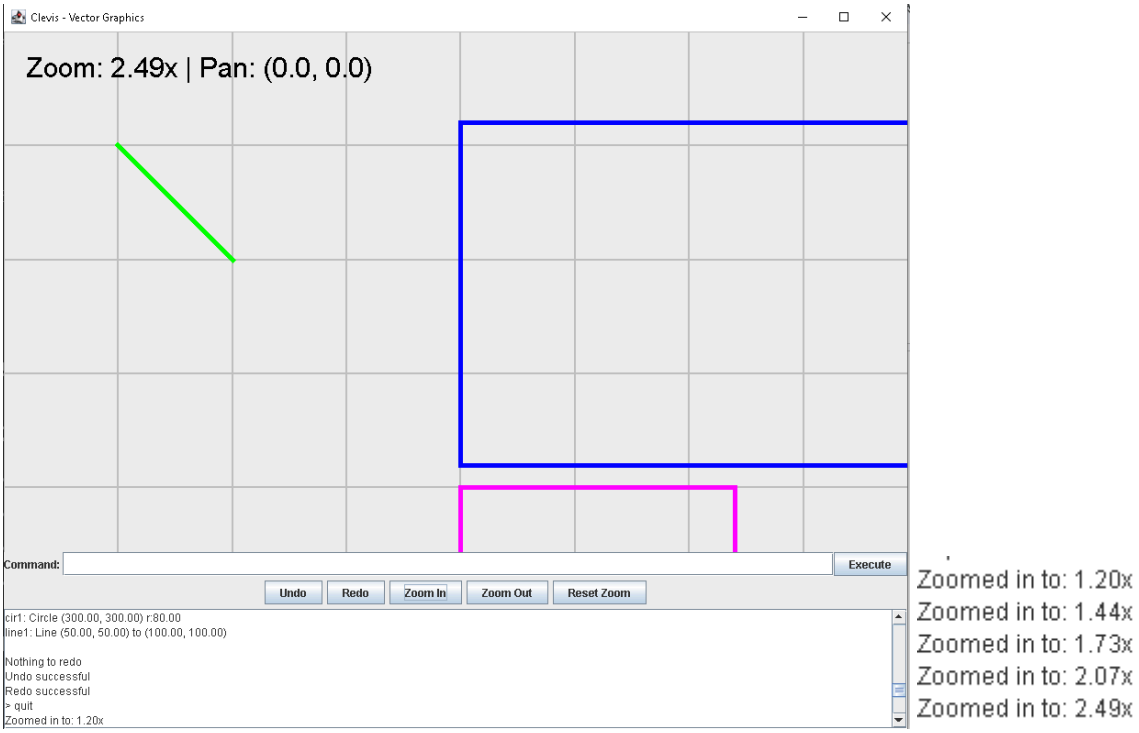
  Redo/Undo (GUI only)
  Redo button: **Reapplies the last undone action**
  Undo button: **Reverts last action**
  Eligible for Undo/Redo: Shape creation, deletion, movement, grouping, ungrouping





  Command: press redo, then undo, then redo
  Result:: Nothing changes (all previous undos are redone), output announcing the function has performed successfully

- GUI Specific Features
  Zoom Functiionality
  command: Press the Zoom in/out button



Command: press zoom in 5 times
Output: Zoom increased from 1.00x zoom to 2.49x zoom

Real time visualisation
Shapes appear immediately upon command execution
canvas is repainted after every operation/command
visual representation matches mathematical coordinates

Command History in both GUI and text file
Any input, wrong or right will be stored in the output bar as history
All inputs stored in text file for record keeping purposes
Any message from the output is cleared when program is exitted

Troubleshooting

<u>Common Error Messages and Solutions</u>:
-“Shape not found” error

Problem: command reference to a non-exist shape/group
Solution:

- check spelling and verify the shape exists using listAll
- check if shape was deleted/ungrouped

-“Shape already exists” error

Problem: creating a shape with repeated name
Solution:

- rename the shape with a unique name
- use unique names for all shapes
- use list all to see existing names as reference

<u>Error messages</u>
-“Usage:[command] parameters…”
Cause:inputting incorrect parameters
Solution:

- provide the parameters according to the command syntax in the pages above
- ensure all parameters have been provided
- ensure that all of the numeric values are valid numbers

-“Invalid numbers”

Cause: non-numeric parameters
Solution:

- check for accidental text in numeric fields
- ensure the format for the decimals is correct, (40.00 instead of 40,00)
- verify that coordinate values are within reasonable range